Full Stack Web Development

Lab Sheet 6— APIs (Practical)

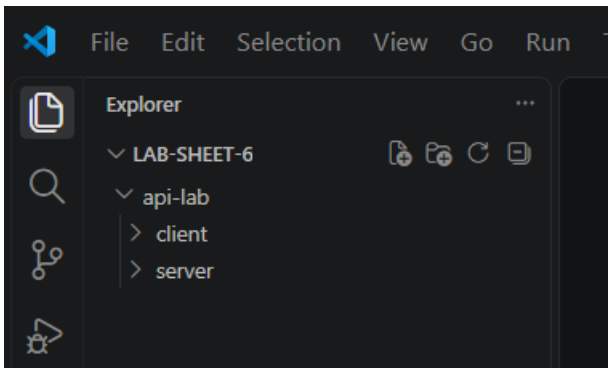
Full Backend Integration: Node.js + Express.js + REST APIs (No Database)

Objective: Build a complete Node.js + Express REST API from scratch, secure and test it, then consume it from both a plain JavaScript client and a React client — covering the full backend-to-frontend pipeline, without a database.

Q1. Server Setup & Building the REST API

(a) Create a folder api-lab with server/ and client/ subfolders. Inside server/, initialize npm, install express, cors, and dotenv, and set up a .env file with PORT=5000 (read via process.env.PORT with a 5000 fallback). Add a "dev" script using nodemon.

A project folder named api-lab was created with two subfolders:
api-lab/
├── client/
└── server/

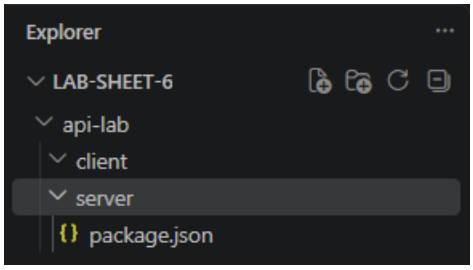


The Node.js project was initialized inside the server folder using npm init -y. This created the package.json file, which stores the project configuration, scripts, dependencies, and other Node.js project information.

```
PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab> cd server
PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab\server> npm init -y
Wrote to C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab\server\package.json:

{
  "name": "server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab\server>
```



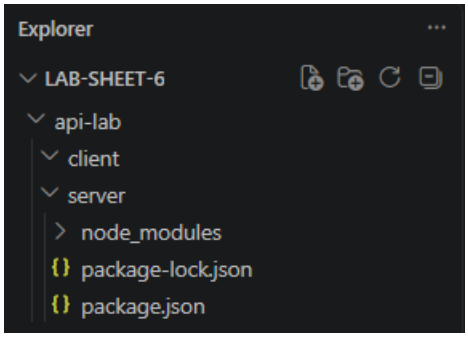
The required backend packages were installed using npm. Express is used to create the web server and REST API, CORS is used to allow communication between the frontend and backend during development, and dotenv is used to load configuration values from the .env file.
Command:
npm install express cors dotenv

```
PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab\server> npm install express cors dotenv

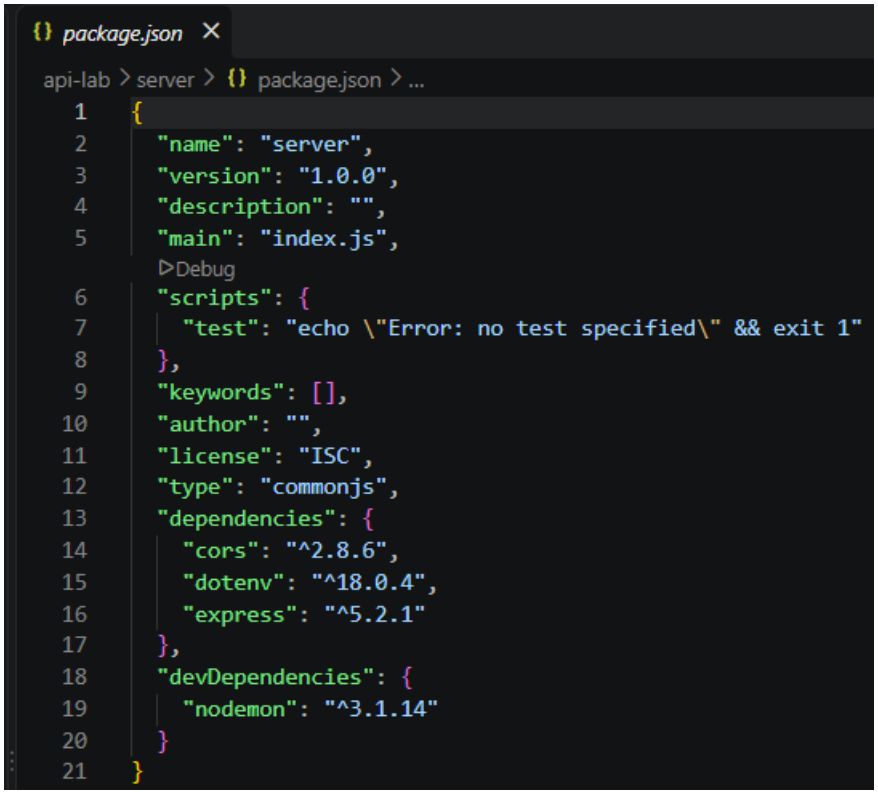
added 71 packages, and audited 72 packages in 3s

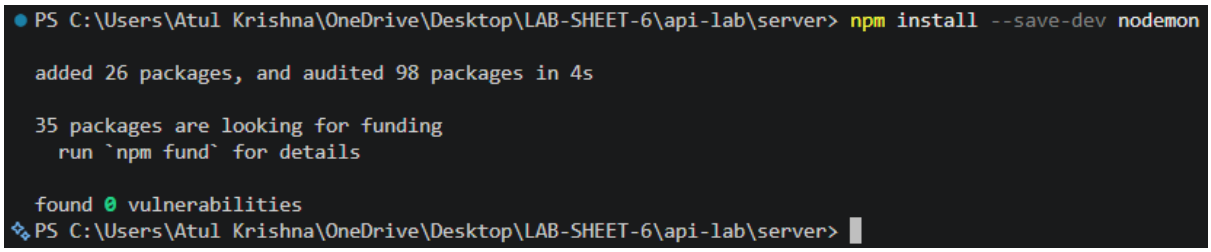
30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab\server>
```

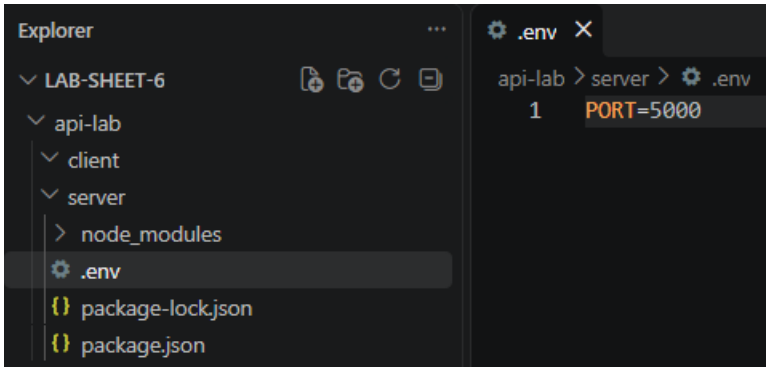


Nodemon was installed as a development dependency. It automatically restarts the Node.js server whenever changes are made to the application files during development.
Command:
npm install --save-dev nodemon



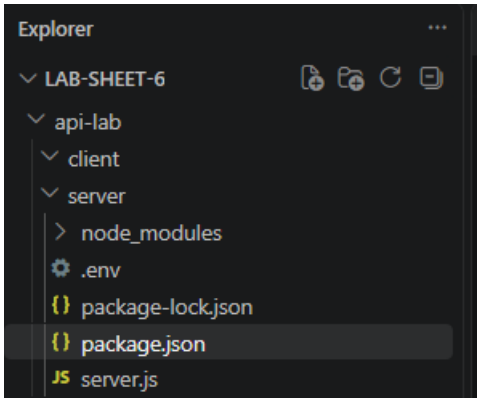


An environment configuration file named .env was created inside the server folder. The server port was stored as an environment variable with the value 5000. Environment variables are used to keep configuration values separate from the application source code.
.env file:
PORT=5000



A server.js file was created using Express.js. The application loads environment variables using dotenv and reads the server port from process.env.PORT. If the environment variable is unavailable, port 5000 is used as the fallback. CORS and JSON parsing middleware were also enabled.
Code:
const express = require("express");
const cors = require("cors");

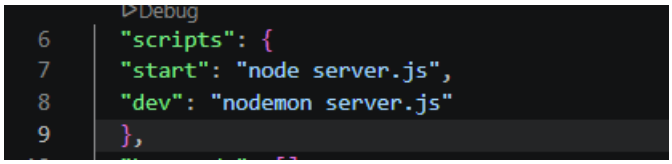
```
require("dotenv").config();
const app = express();
const PORT = process.env.PORT || 5000;
app.use(cors());
app.use(express.json());
app.get("/", (req, res) => {
  res.json({
    message: "Task Manager API is running"
  });
});
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```



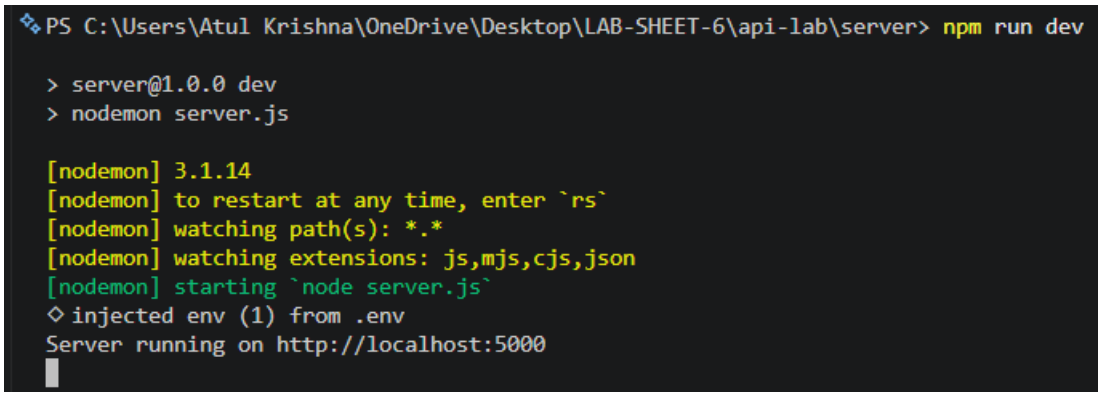
Adding the Development Script
Code:

```
"scripts": {
 "start": "node server.js",
 "dev": "nodemon server.js"
}
```

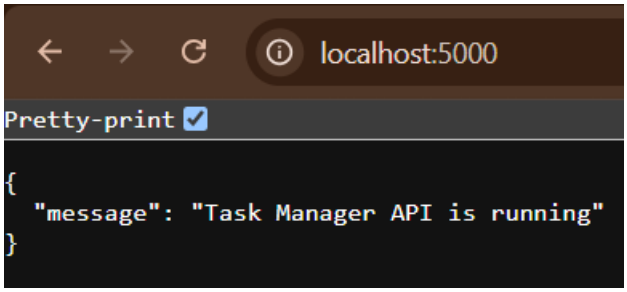
The dev script uses Nodemon so that the server automatically restarts whenever changes are made during development.



npm run dev



http://localhost:5000

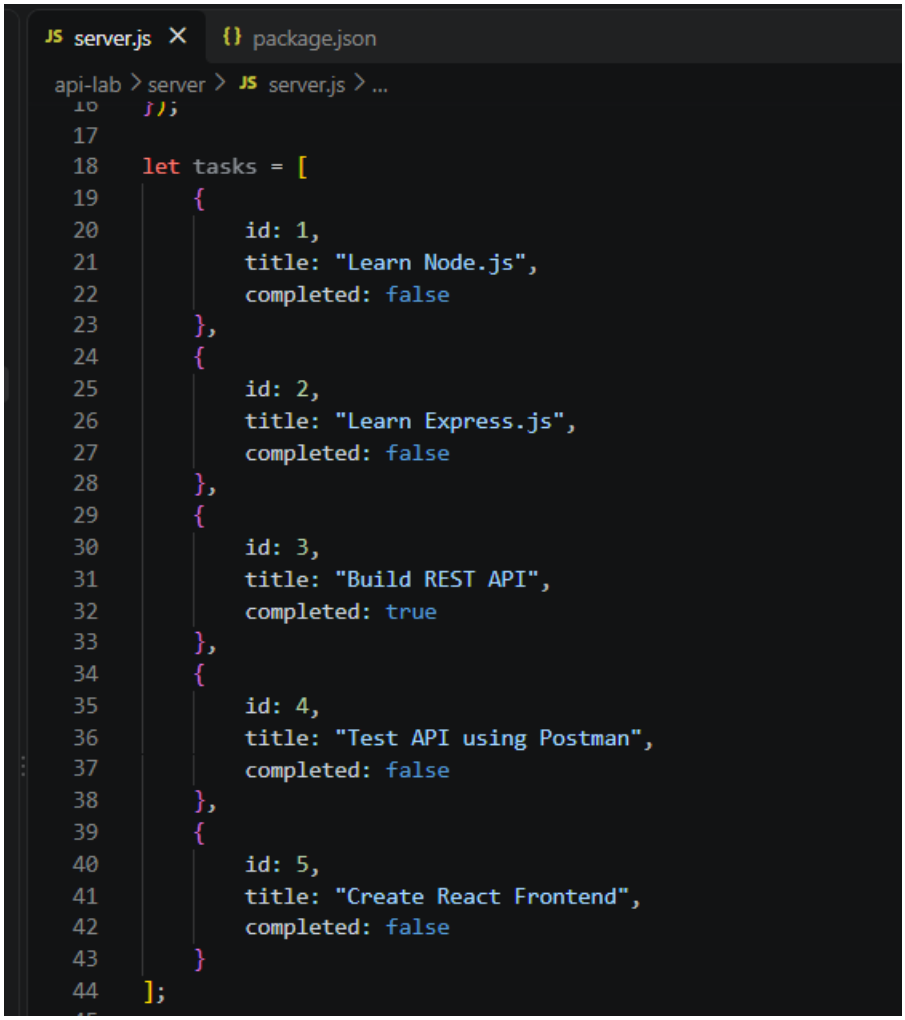


The Node.js and Express.js server was successfully configured and executed on port 5000 using an environment variable. Nodemon was configured for development, and the server was successfully tested through the browser.

(b) Using an in-memory array of at least 5 “task” objects (id, title, completed), build all 5 CRUD routes: GET /api/tasks, GET /api/tasks/:id, POST /api/tasks, PUT /api/tasks/:id, and DELETE /api/tasks/:id — with a proper 404 JSON error when an id doesn't exist.

An in-memory array named tasks was created to store task objects without using a database. Each task contains three properties: id, title, and completed. Five initial task records were added to the array.
Code:

```
let tasks = [
  {
    id: 1,
    title: "Learn Node.js",
    completed: false
  },
  {
    id: 2,
    title: "Learn Express.js",
    completed: false
  },
  {
    id: 3,
    title: "Build REST API",
    completed: true
  },
  {
    id: 4,
    title: "Test API using Postman",
    completed: false
  },
  {
    id: 5,
    title: "Create React Frontend",
    completed: false
  }
];
```

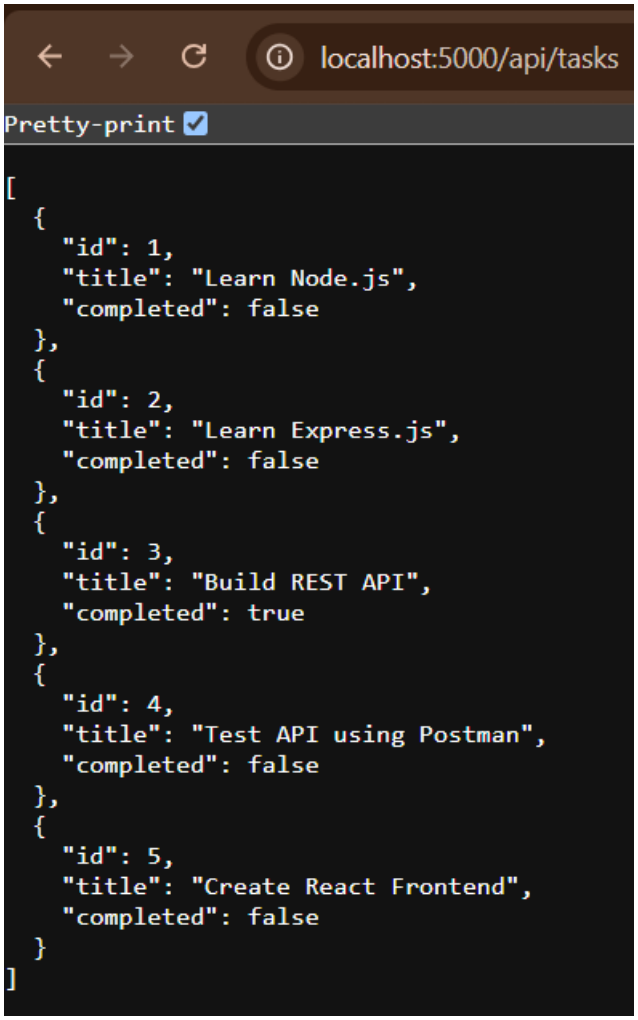


GET /api/tasks — Retrieve All Tasks
This route retrieves all task objects from the in-memory array. The res.json() method sends the task array to the client in JSON format.

Code:

```
app.get("/api/tasks", (req, res) => {
  res.json(tasks);
});
```

Endpoint:
GET http://localhost:5000/api/tasks



GET /api/tasks/:id — Retrieve a Single Task

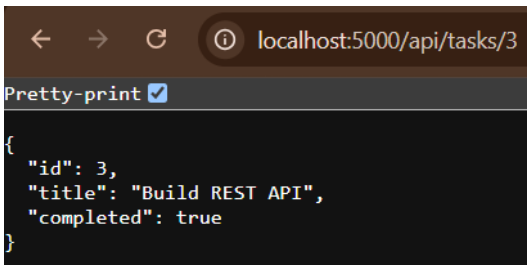
This route retrieves a specific task using its ID. The ID is obtained from the URL parameter and converted into an integer using parseInt(). The find() method searches for the task in the in-memory array. If the task does not exist, the API returns a JSON error with HTTP status code 404.

Code:

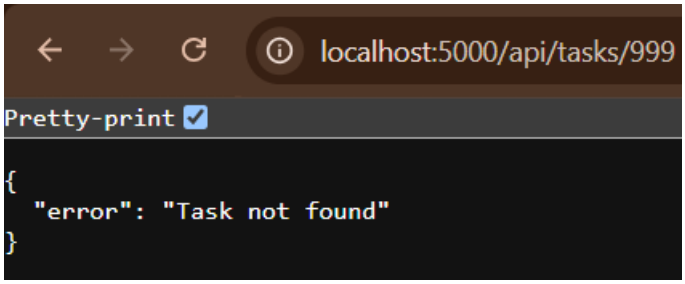
```
app.get("/api/tasks/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const task = tasks.find(task => task.id === id);
  if (!task) {
    return res.status(404).json({
      error: "Task not found"
    });
  }
  res.json(task);
});
```

Endpoints tested:

GET http://localhost:5000/api/tasks/3



GET http://localhost:5000/api/tasks/999



POST /api/tasks — Create a New Task
This route creates a new task using the data received in the request body. A unique ID is generated for the new task and the task is added to the in-memory array. The API returns the newly created task with HTTP status code 201.

Code:

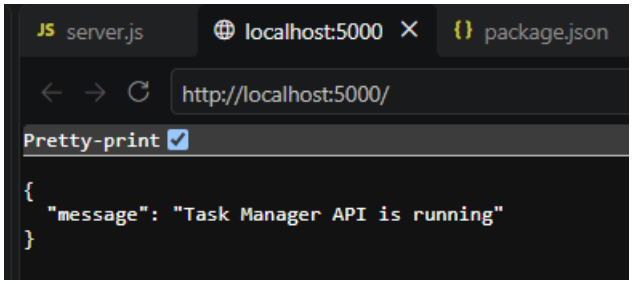
```
app.post("/api/tasks", (req, res) => {
  const { title, completed } = req.body;
  const newTask = {
    id: tasks.length > 0 ? Math.max(...tasks.map(task => task.id)) + 1 : 1,
    title: title,
    completed: completed ?? false
  };
  tasks.push(newTask);
  res.status(201).json(newTask);
});
```

Test Request

```
POST /api/tasks
{
  "title": "Learn REST APIs",
  "completed": false
}
```

Expected Result

```
{
  "id": 6,
  "title": "Learn REST APIs",
  "completed": false
}
```



PUT /api/tasks/:id — Update a Task
This route updates an existing task using its ID. The title and completed status are obtained from the request body. If the requested task ID does not exist, the API returns a JSON 404 error.

Code:

```
app.put("/api/tasks/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const task = tasks.find(task => task.id === id);
  if (!task) {
    return res.status(404).json({
      error: "Task not found"
    });
  }
```

```
  const { title, completed } = req.body;
  task.title = title ?? task.title;
  task.completed = completed ?? task.completed;
  res.json(task);
});
```

```
app.put("/api/tasks/:id", (req, res) => {
    const id = parseInt(req.params.id);

    const task = tasks.find(task => task.id === id);

    if (!task) {
        return res.status(404).json({
            error: "Task not found"
        });
    }

    const { title, completed } = req.body;

    task.title = title ?? task.title;
    task.completed = completed ?? task.completed;

    res.json(task);
});
```

DELETE /api/tasks/:id:
This route deletes a task using its ID. If the task does not exist, it returns a 404 JSON error. Otherwise, the selected task is removed from the in-memory array and a success message is returned.

```
app.delete("/api/tasks/:id", (req, res) => {
    const id = parseInt(req.params.id);

    const index = tasks.findIndex(task => task.id === id);

    if (index === -1) {
        return res.status(404).json({
            error: "Task not found"
        });
    }

    const deletedTask = tasks.splice(index, 1)[0];

    res.json({
        message: "Task deleted successfully",
        task: deletedTask
    });
});
```

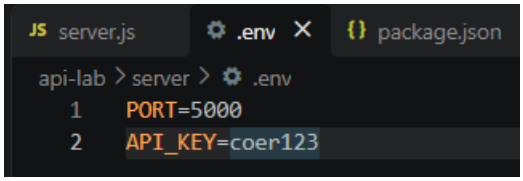
(c) Write a custom logger middleware (method, URL, timestamp) applied globally, and an auth-check middleware requiring a header x-api-key (stored in .env) applied only to the POST, PUT, and DELETE routes.

A custom logger middleware is created to record the timestamp, HTTP method, and requested URL of every incoming request. It is applied globally using app.use(logger). The next() function passes the request to the next middleware or route.

```
const logger = (req, res, next) => {
    console.log(
        `[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`
    );
    next();
};

app.use(logger);
```

The API key is stored in the .env file so that the secret value is not directly written inside the application code. The authentication middleware will compare the x-api-key request header with this environment variable.
PORT=5000
API_KEY=coer123



The authentication middleware checks the x-api-key header of an incoming request and compares it with the API key stored in the .env file. If the key is missing or invalid, the server returns a 401 Unauthorized response. Otherwise, the request is passed to the next middleware or route using next().

```
const authCheck = (req, res, next) => {
    const apiKey = req.headers["x-api-key"];

    if (!apiKey || apiKey !== process.env.API_KEY) {
        return res.status(401).json({
            error: "Unauthorized: Invalid or missing API key"
        });
    }

    next();
};
```

The authentication middleware is applied only to POST, PUT, and DELETE routes. Therefore, these operations require a valid x-api-key header, while GET requests remain publicly accessible.

GET /api/tasks → No API key required
GET /api/tasks/:id → No API key required
POST /api/tasks → API key required
PUT /api/tasks/:id → API key required
DELETE /api/tasks/:id → API key required

```
app.post("/api/tasks", authCheck, (req, res) => {
  // POST code
});
app.put("/api/tasks/:id", authCheck, (req, res) => {
  // PUT code
});
app.delete("/api/tasks/:id", authCheck, (req, res) => {
  // DELETE code
});
```

```
app.post("/api/tasks", authCheck, (req, res) => {
```

```
app.put("/api/tasks/:id", authCheck, (req, res) => {
```

```
app.delete("/api/tasks/:id", authCheck, (req, res) => {
```

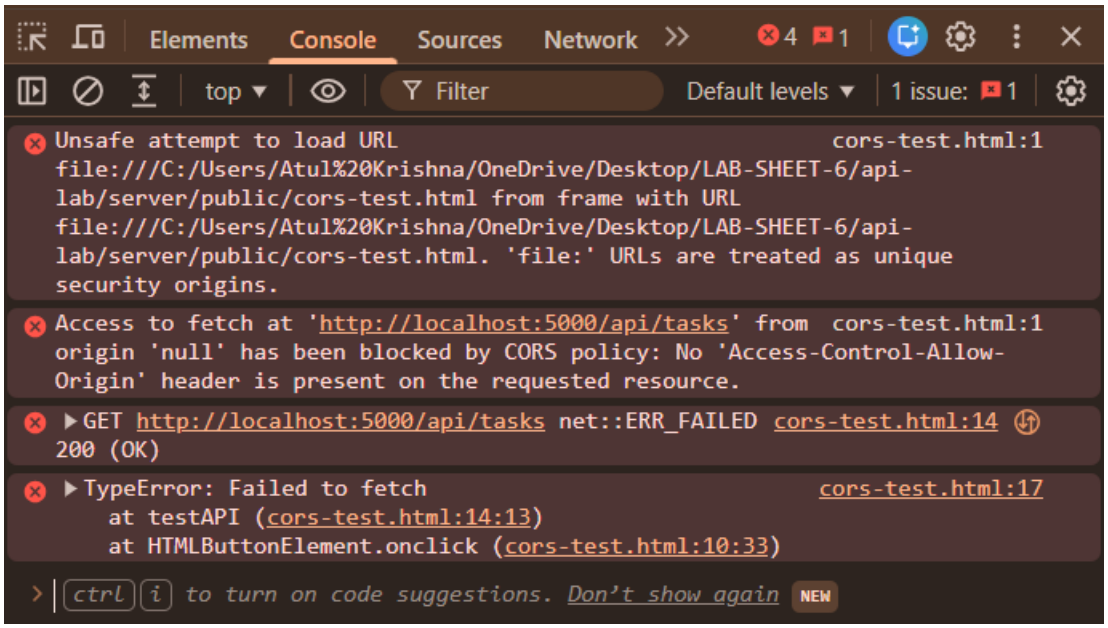
```
app.use(cors());
app.use(express.json());
```

```
const logger = (req, res, next) => {
  console.log(
    `[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`
  );
  next();
};
app.use(logger);
const authCheck = (req, res, next) => {
  const apiKey = req.headers["x-api-key"];
  if (!apiKey || apiKey !== process.env.API_KEY) {
    return res.status(401).json({
      error: "Unauthorized: Invalid or missing API key"
    });
  }
  next();
};
```

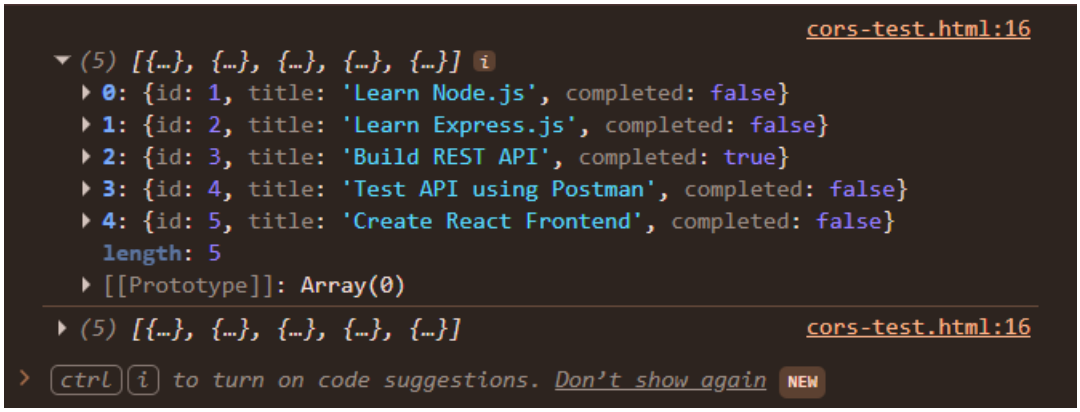
Q2. Securing, Testing & Error-Handling the API

(a) Enable CORS for development. Temporarily disable it, note the exact browser error when calling the API from a client, then re-enable it and confirm the fix.

CORS Disabled Result:
When CORS was temporarily disabled, the browser blocked the request from the client to the API. The browser console displayed: “Access to fetch at 'http://localhost:5000/api/tasks' from origin 'null' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.”
Error observed:
GET http://localhost:5000/api/tasks net::ERR_FAILED 200 (OK)
TypeError: Failed to fetch



After re-enabling CORS using app.use(cors()), the browser was able to access the API successfully. The CORS error disappeared and the /api/tasks endpoint returned the task data to the client.



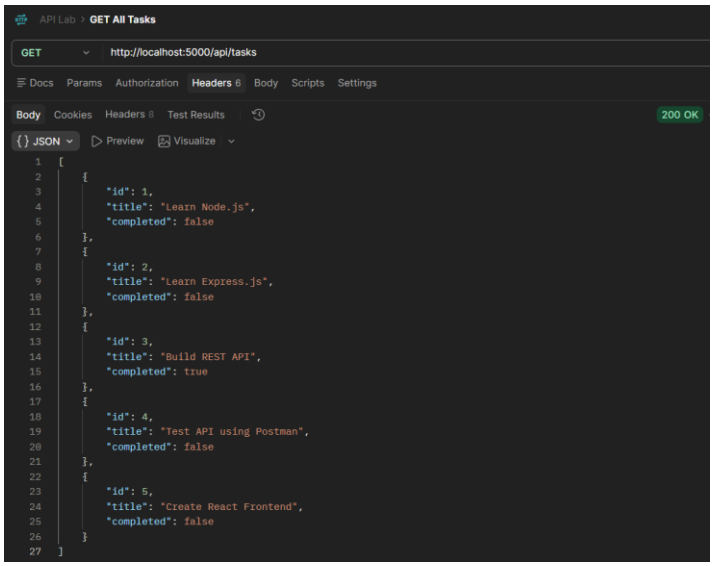


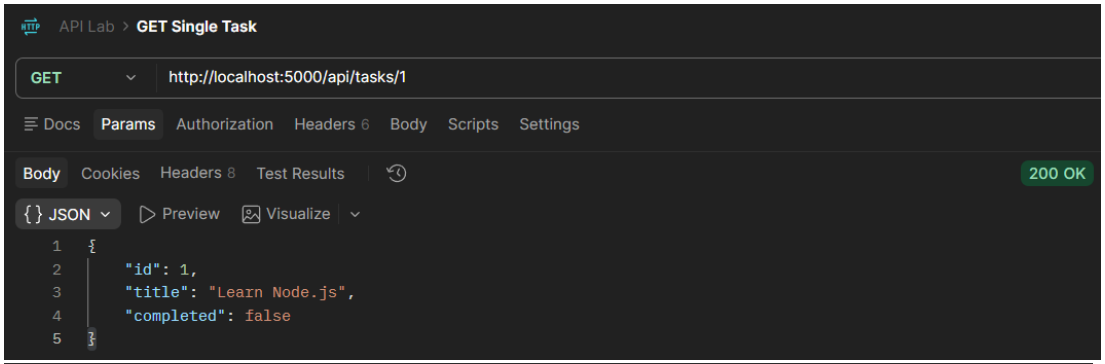
CORS Test

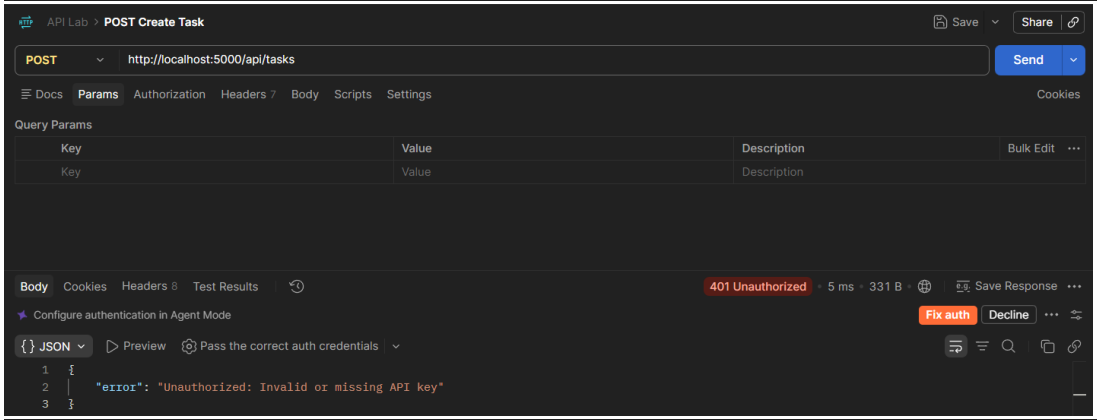
Call API

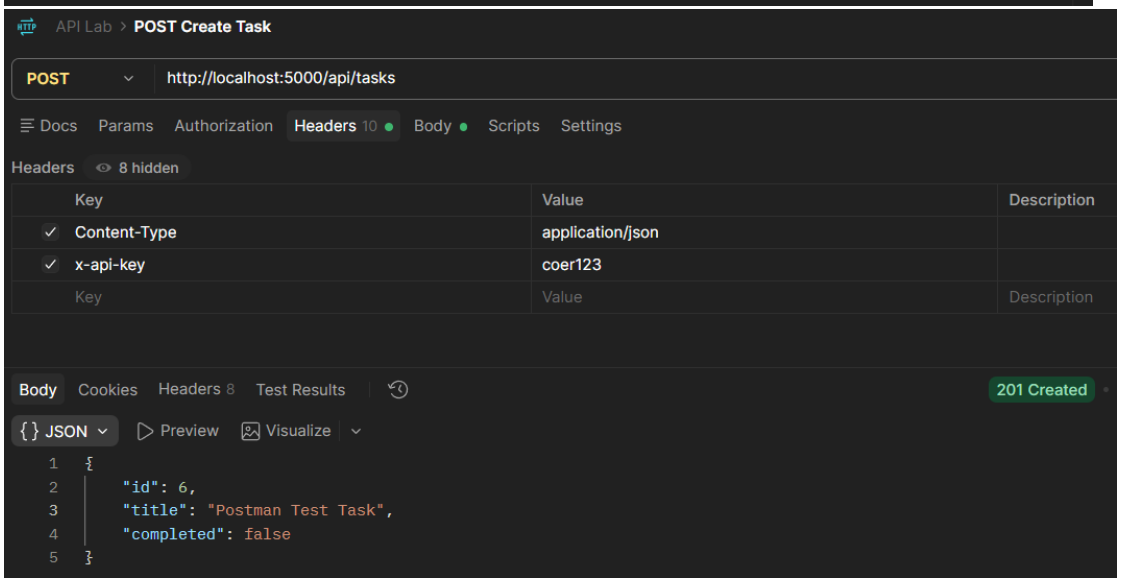
```
PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab\server> npm run dev
[nodemon] starting `node server.js`
◇ injected env (2) from .env
Server running on http://localhost:5000
[2026-09-29T19:25:25.581Z] GET /api/tasks
[2026-09-29T19:25:54.079Z] GET /api/tasks
[2026-09-29T19:33:25.016Z] GET /api/tasks
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (2) from .env
Server running on http://localhost:5000
[2026-09-29T19:44:28.328Z] GET /api/tasks
[2026-09-29T19:44:31.826Z] GET /api/tasks
```

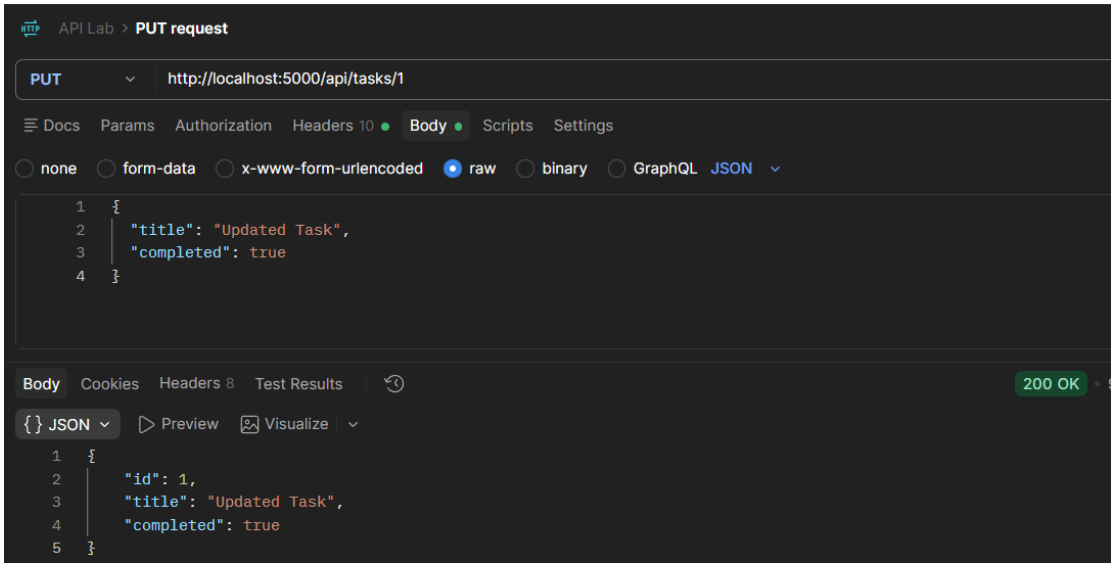
(b) Create a Postman (or Thunder Client) collection named "API Lab" covering all 5 endpoints, including the x-api-key header where required. Confirm GET works without the key but POST returns 401 without it. Screenshot one successful POST request (body + response).
CORS-enabled API response/output shown in the browser/Postman.

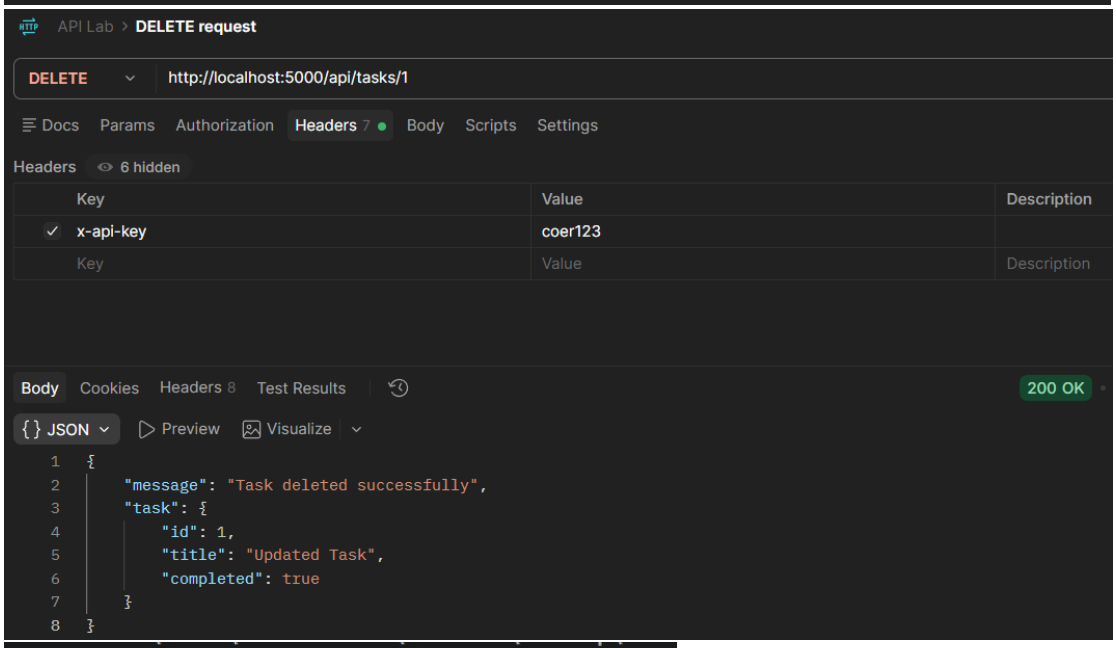


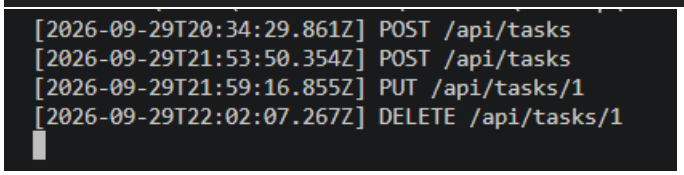

API Lab > GET Single Task
GET
http://localhost:5000/api/tasks/1
Docs Params Authorization Headers 6 Body Scripts Settings
Body Cookies Headers 8 Test Results
200 OK
{ } JSON Preview Visualize
{
"id": 1,
"title": "Learn Node.js",
"completed": false
}


API Lab > POST Create Task
Save Share
POST
http://localhost:5000/api/tasks
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Query Params
Key Value Description Bulk Edit
Key Value Description
Body Cookies Headers 8 Test Results
401 Unauthorized 5 ms 331 B Save Response
Configure authentication in Agent Mode
Fix auth Decline
{ } JSON Preview Pass the correct auth credentials
{
"error": "Unauthorized: Invalid or missing API key"
}


API Lab > POST Create Task
POST
http://localhost:5000/api/tasks
Docs Params Authorization Headers 10 Body Scripts Settings
Headers 8 hidden
Key Value Description
Content-Type application/json
x-api-key coer123
Key Value Description
Body Cookies Headers 8 Test Results
201 Created
{ } JSON Preview Visualize
{
"id": 6,
"title": "Postman Test Task",
"completed": false
}







(c) Add a 404 handler (after all routes) for unmatched URLs, and a centralized error-handling middleware (4 parameters) that returns a consistent JSON error shape for any thrown error.

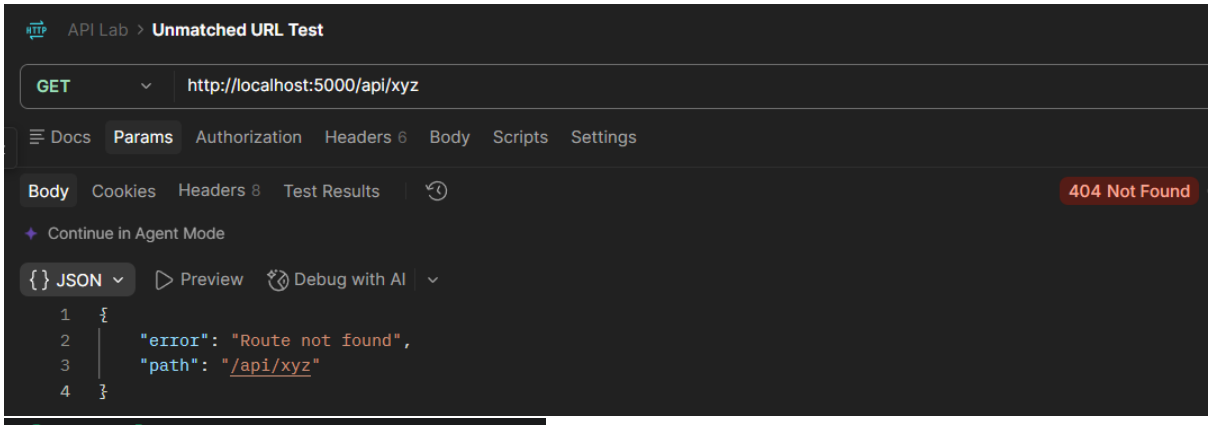
```
// 404 handler
app.use((req, res, next) => {
  res.status(404).json({
    error: "Route not found",
    path: req.originalUrl
  });
});
// Centralized error-handling middleware
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({
    error: "Internal Server Error",
    message: err.message
  });});
```

```
// 404 handler
app.use((req, res, next) => {
    res.status(404).json({
        error: "Route not found",
        path: req.originalUrl
    });
});

// Centralized error-handling middleware
app.use((err, req, res, next) => {
    console.error(err);

    res.status(500).json({
        error: "Internal Server Error",
        message: err.message
    });
});
```

404 handler for unmatched URL

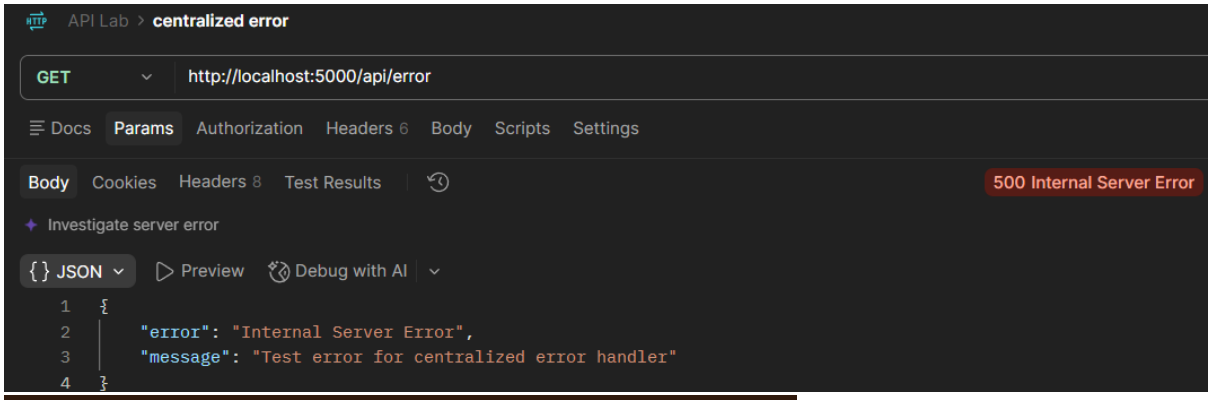


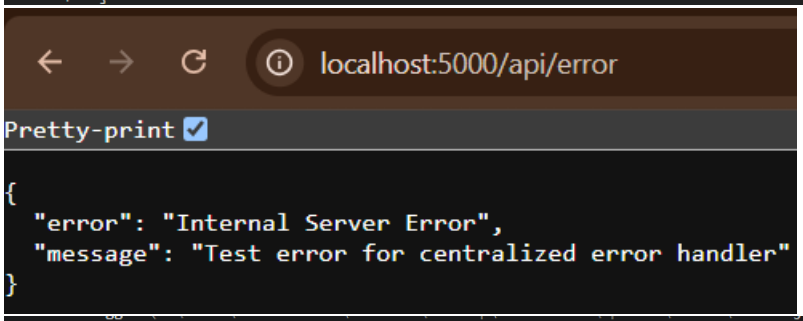

```
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (2) from .env
Server running on http://localhost:5000
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (2) from .env
Server running on http://localhost:5000
[2026-09-29T22:12:09.786Z] GET /
[2026-09-29T22:14:27.904Z] GET /api/xyz
```

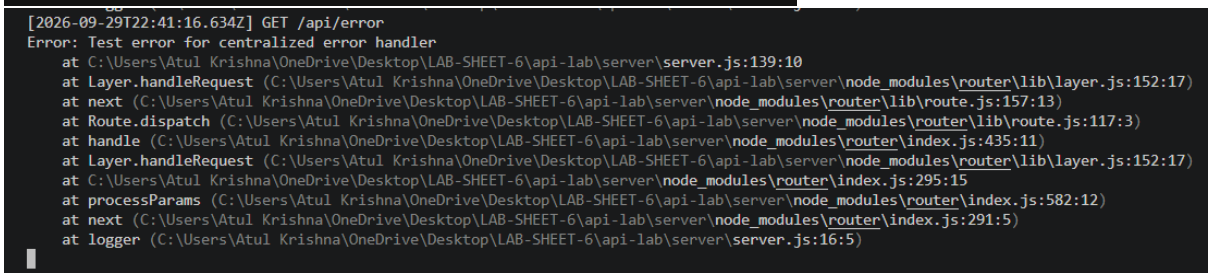
// Test route for centralized error handler
app.get("/api/error", (req, res, next) => {
  next(new Error("Test error for centralized error handler"));
});

```
// Test route for centralized error handler
app.get("/api/error", (req, res, next) => {
    next(new Error("Test error for centralized error handler"));
});

// 404 handler
```







Q3. Consuming the API — Plain JavaScript Client

(a) Inside client/, create a simple index.html and script.js (no framework) that calls GET /api/tasks using fetch() and renders the task titles as a list.

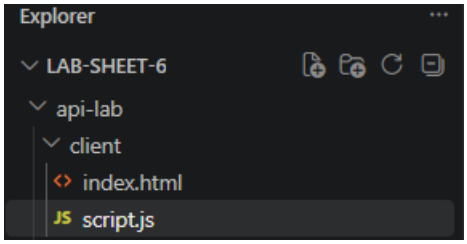
```
api-lab
├── server
└── client
```

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Task List</title>
</head>
<body>
  <h2>Tasks</h2>
  <ul id="taskList"></ul>
  <script src="script.js"></script>
</body>
</html>
```

script.js

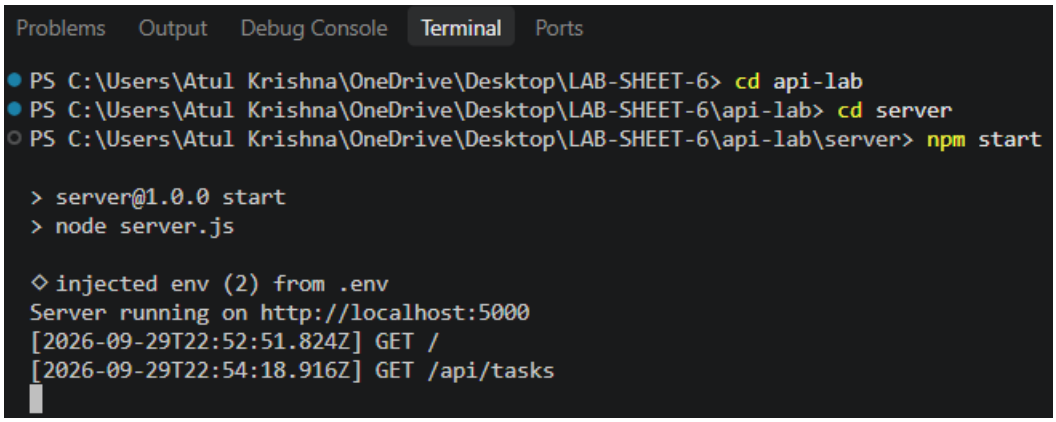
```
fetch("http://localhost:5000/api/tasks")
  .then(response => response.json())
  .then(tasks => {
    const taskList = document.getElementById("taskList");
    tasks.forEach(task => {
      const li = document.createElement("li");
      li.textContent = task.title;
      taskList.appendChild(li);
    });
  })
  .catch(error => {
    console.error("Error:", error);});
```





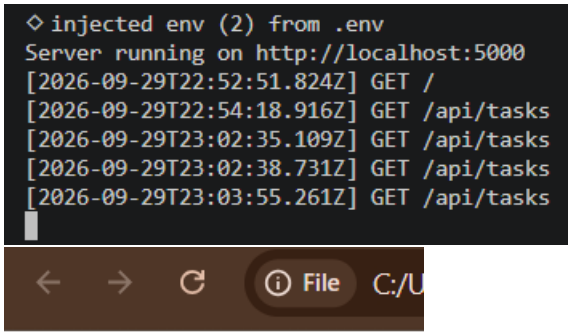
Tasks

- Learn Node.js
- Learn Express.js
- Build REST API
- Test API using Postman
- Create React Frontend



(b) Add loading and error handling in plain JS: show “Loading...” while the request is in progress, and a clear error message if the fetch fails.

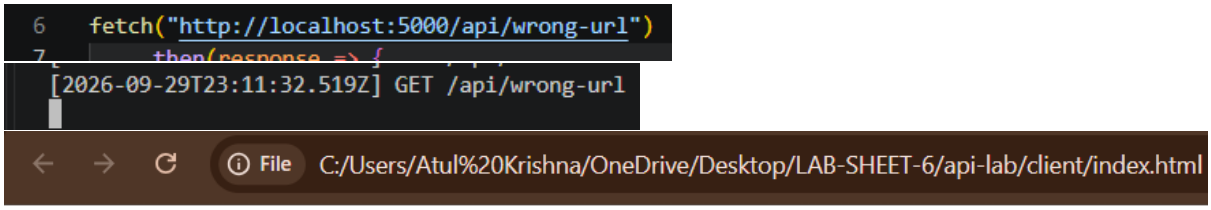
```
const taskList = document.getElementById("taskList");
// Show loading message
taskList.innerHTML = "<li>Loading...</li>";
fetch("http://localhost:5000/api/tasks")
  .then(response => {
    if (!response.ok) {
      throw new Error("Failed to fetch tasks");
    }
    return response.json();
  })
  .then(tasks => {
    taskList.innerHTML = "";
    tasks.forEach(task => {
      const li = document.createElement("li");
      li.textContent = task.title;
      taskList.appendChild(li);
    });
  })
  .catch(error => {
    taskList.innerHTML = "<li>Error: Unable to load tasks.</li>";
    console.error(error);
  });
```



Tasks

- Learn Node.js
- Learn Express.js
- Build REST API
- Test API using Postman
- Create React Frontend

fetch("http://localhost:5000/api/wrong-url")
Loading and error handling implemented in plain JavaScript

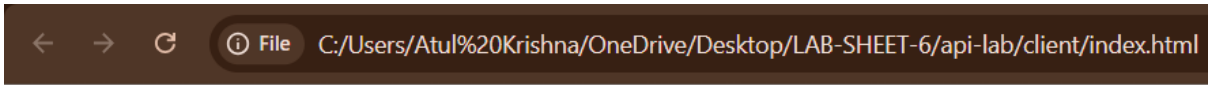


Tasks

- Error: Unable to load tasks.

(c) Test the failure path by stopping the server while the page is open, and confirm the error message appears correctly instead of the page breaking.

Error handling when the API server is stopped
Server stopped → error message displayed without page breaking

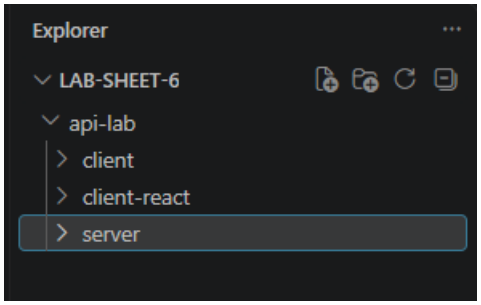


Tasks

- Error: Unable to load tasks.

Q4. Consuming the API — React Client

(a) Create a React app (Vite recommended) inside client-react/. Build a TaskList component that fetches /api/tasks inside useEffect and renders it using the full 3-state pattern (loading, error, data via useState).
React TaskList component fetching and displaying tasks from the REST API
npm create vite@latest client-react -- --template react
api-lab
├── server
├── client
└── client-react



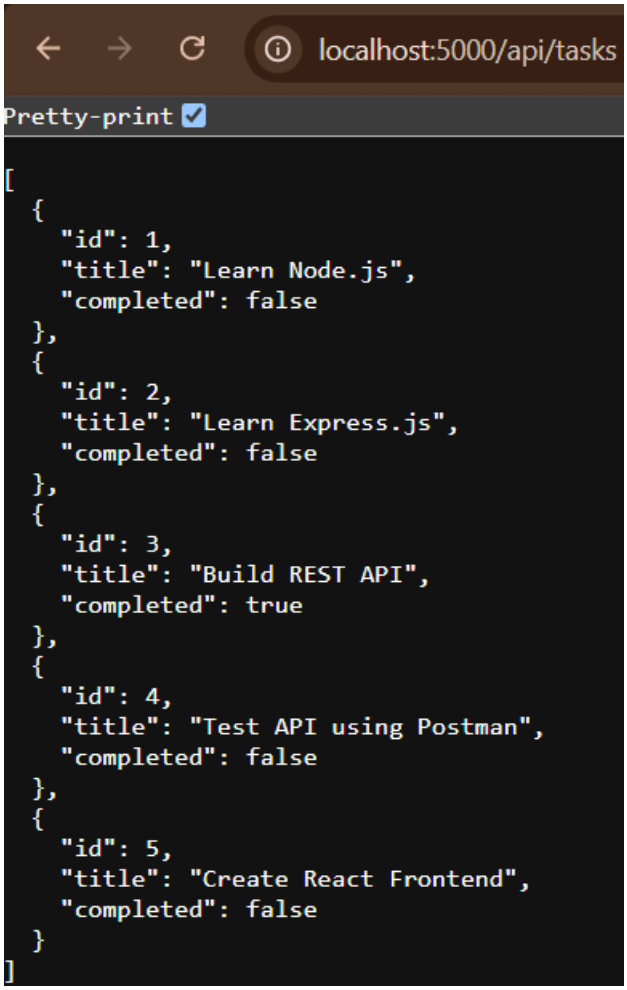
TaskList.jsx

```
import { useEffect, useState } from "react";
function TaskList() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  useEffect(() => {
    fetch("http://localhost:5000/api/tasks")
      .then(response => {
        if (!response.ok) {
          throw new Error("Failed to fetch tasks");
        }
        return response.json();
      })
      .then(data => {
        setTasks(data);
        setLoading(false);
      })
      .catch(error => {
        setError(error.message);
        setLoading(false);
      });
  }, []);
  if (loading) {
    return <p>Loading...</p>;
  }
  if (error) {
    return <p>Error: {error}</p>;
  }
  return (
    <div>
      <h2>Tasks</h2>
      <ul>
        {tasks.map(task => (
          <li key={task.id}>{task.title}</li>
        ))}
      </ul>
    </div>
  );
}
export default TaskList;
```

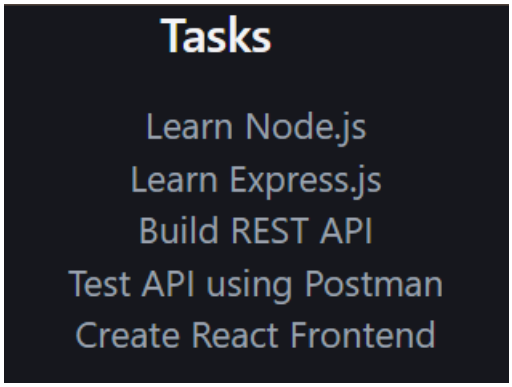
App.jsx

```
import TaskList from "./TaskList";
function App() {
  return (
    <TaskList />
  );
}
export default App;
```

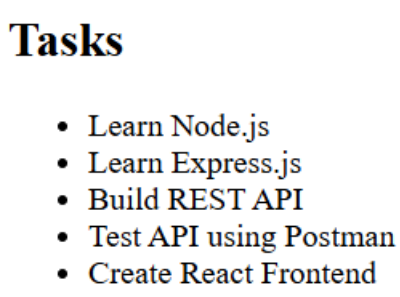
http://localhost:5000/api/tasks



http://localhost:5173/



client/index.html



(b) Rewrite the same fetch call using axios instead, and note in a short comment how the code differs (JSON parsing, error handling).
npm install axios

```
PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6> cd api-lab
PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab> cd client-react
PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab\client-react> npm install axios

added 25 packages, and audited 169 packages in 5s

37 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Atul Krishna\OneDrive\Desktop\LAB-SHEET-6\api-lab\client-react>
```

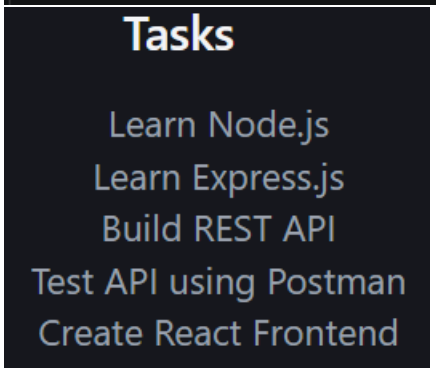
import axios from "axios";

api-lab > client-react > src > TaskList.jsx > TaskList > useEffect() callback

```
1  import { useEffect, useState } from "react";
2  import axios from "axios";
```

```
useEffect(() => {
  axios.get("http://localhost:5000/api/tasks")
    .then(response => {
      setTasks(response.data);
      setLoading(false);
    })
    .catch(error => {
      setError(error.message);
      setLoading(false);
    });
}, []);
// Axios automatically parses JSON and handles failed HTTP responses,
// while fetch needs response.json() and manual response.ok checking.
```

```
useEffect(() => {
axios.get("http://localhost:5000/api/tasks")
    .then(response => {
        setTasks(response.data);
        setLoading(false);
    })
    .catch(error => {
        setError(error.message);
        setLoading(false);
    });
}, []);
```



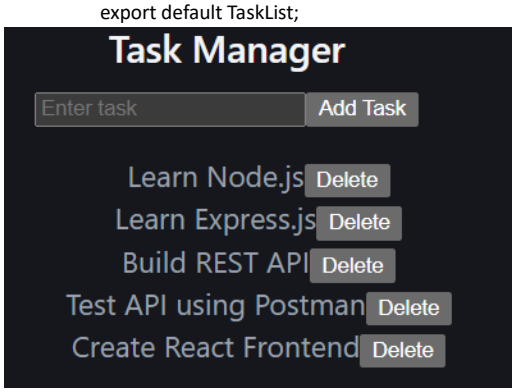
(c) Build a form that POSTs a new task (with the Content-Type and x-api-key headers, and JSON.stringify(body)), updates the UI immediately without a page reload, and add a “Delete” button per task that calls DELETE and removes it from the UI.

TaskList.jsx

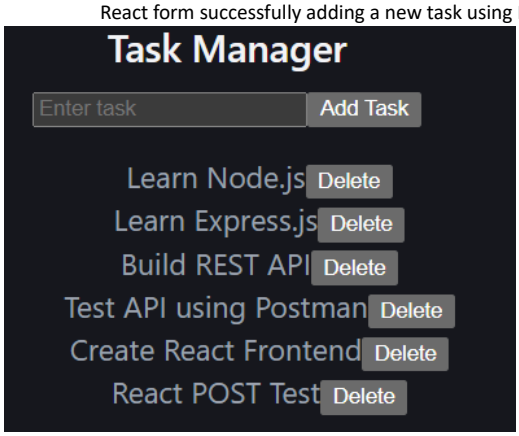
```
import { useEffect, useState } from "react";
function TaskList() {
  const [tasks, setTasks] = useState([]);
  const [title, setTitle] = useState("");
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const API_KEY = "coer123";
  useEffect(() => {
```

```
  fetch("http://localhost:5000/api/tasks")
      .then(response => response.json())
      .then(data => {
        setTasks(data);
        setLoading(false);
      })
      .catch(() => {
        setError("Failed to fetch tasks");
        setLoading(false);
      });
}, []);
// Add new task
const addTask = (e) => {
  e.preventDefault();
  const body = {
    title: title,
    completed: false
  };
  fetch("http://localhost:5000/api/tasks", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-api-key": API_KEY
    },
    body: JSON.stringify(body)
  })
    .then(response => response.json())
    .then(newTask => {
      setTasks([...tasks, newTask]);
      setTitle("");
    })
    .catch(() => {
      setError("Failed to add task");
    });
};
// Delete task
const deleteTask = (id) => {
  fetch(`http://localhost:5000/api/tasks/${id}`, {
    method: "DELETE",
    headers: {
      "x-api-key": API_KEY
    }
  })
    .then(response => response.json())
    .then(() => {
      setTasks(tasks.filter(task => task.id !== id));
    })
    .catch(() => {
      setError("Failed to delete task");
    });
};
if (loading) {
  return <p>Loading...</p>;
}
if (error) {
  return <p>Error: {error}</p>;
}
return (
  <div>
    <h2>Task Manager</h2>
    <form onSubmit={addTask}>
      <input
        type="text"
        placeholder="Enter task"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
```

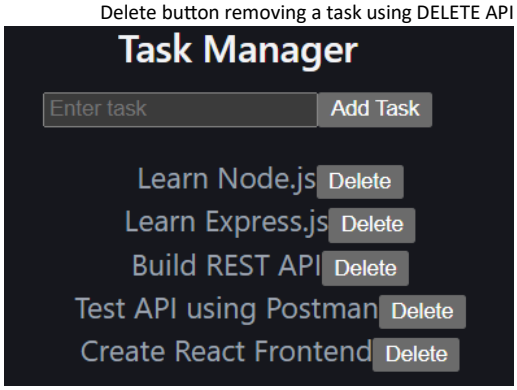
```
        />
        <button type="submit">Add Task</button>
      </form>
      <ul>
        {tasks.map(task => (
          <li key={task.id}>
            {task.title}
            <button onClick={() => deleteTask(task.id)}>
              Delete
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}
export default TaskList;
```



React form successfully adding a new task using POST API



Delete button removing a task using DELETE API without page reload



Q5. Full-Stack Error Handling, Config & Final Integration

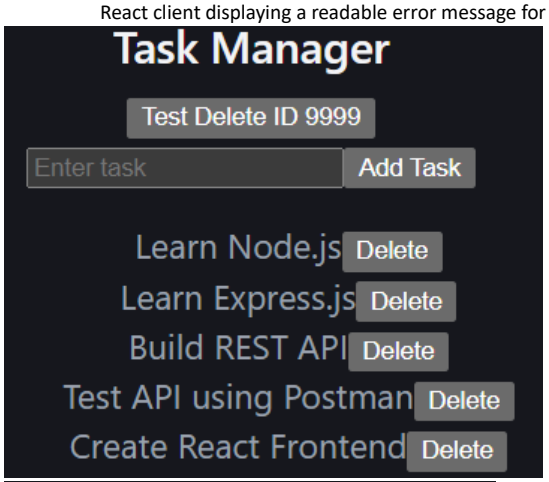
(a) On the client, make sure a 404 or 401 response from the server shows a proper, readable error message in the UI (not a crash or blank screen). Test this by deleting a task using a fake id (e.g. 9999) from the React UI and confirm both server and client respond gracefully.

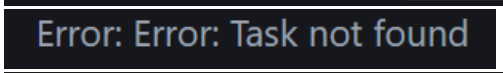
deleteTask

```
const deleteTask = (id) => {
```

```
  fetch(`http://localhost:5000/api/tasks/${id}`, {
    method: "DELETE",
    headers: {
      "x-api-key": API_KEY
    }
  })
    .then(async response => {
      const data = await response.json();
      if (!response.ok) {
        throw new Error(data.error || "Failed to delete task");
      }
      return data;
    })
    .then(() => {
      setTasks(tasks.filter(task => task.id !== id));
    })
    .catch(error => {
      setError(`Error: ${error.message}`);
    });
};
```

React client displaying a readable error message for a 404 response



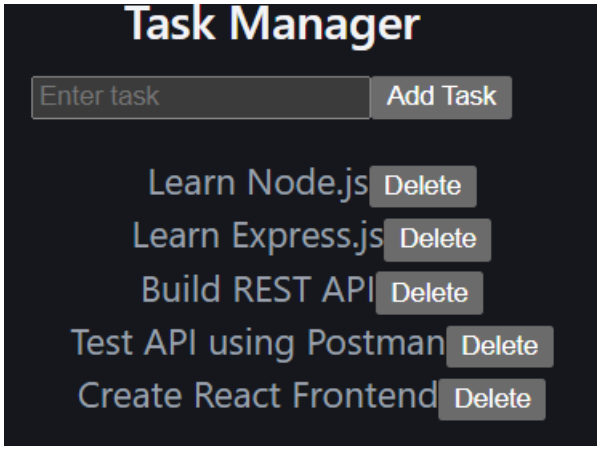



```
[2026-09-30T05:12:59.973Z] DELETE /api/tasks/6
[2026-09-30T05:23:32.431Z] GET /api/tasks
[2026-09-30T05:23:32.433Z] GET /api/tasks
[2026-09-30T05:23:34.918Z] DELETE /api/tasks/9999
[2026-09-30T05:23:43.612Z] GET /api/tasks
[2026-09-30T05:23:43.614Z] GET /api/tasks
[2026-09-30T05:23:45.310Z] DELETE /api/tasks/9999
[2026-09-30T05:23:46.945Z] GET /api/tasks
[2026-09-30T05:23:46.947Z] GET /api/tasks
[2026-09-30T05:23:48.764Z] DELETE /api/tasks/9999
[2026-09-30T05:23:50.077Z] GET /api/tasks
[2026-09-30T05:23:50.080Z] GET /api/tasks
[2026-09-30T05:23:54.801Z] DELETE /api/tasks/9999
[2026-09-30T05:23:56.482Z] GET /api/tasks
[2026-09-30T05:23:56.483Z] GET /api/tasks
[2026-09-30T05:24:26.277Z] DELETE /api/tasks/9999
[2026-09-30T05:29:00.438Z] GET /api/tasks
[2026-09-30T05:29:00.441Z] GET /api/tasks
```

```
    <h2>Task Manager</h2>

    <button onClick={() => deleteTask(9999)}>
        Test Delete ID 9999
    </button>
```

After refresh http://localhost:5173/
Testing is complete

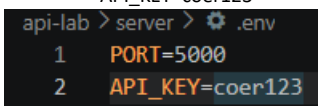


(b) Move every hardcoded value into config: PORT and the API key into the server's .env (confirmed in .gitignore), and the API's base URL into a React environment variable (e.g. VITE_API_URL) instead of a hardcoded localhost address.

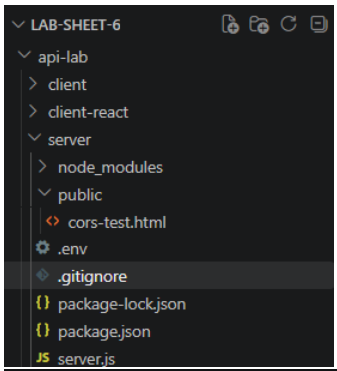
Server and React configuration values moved to environment variables

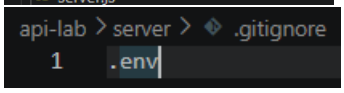
PORT=5000

API_KEY=coer123

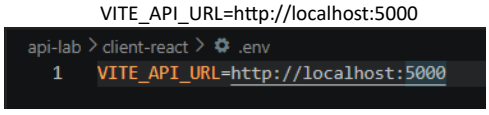


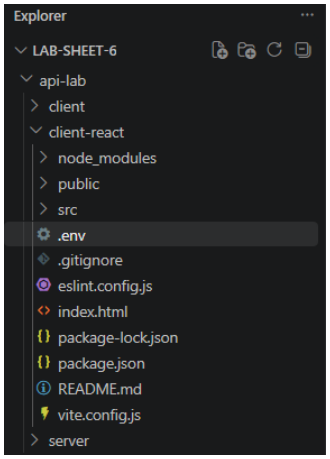
server/
├── .env
├── .gitignore
├── server.js
└── ...



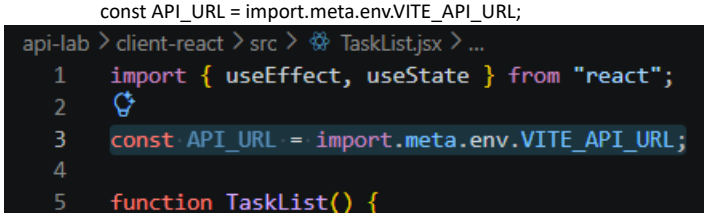


VITE_API_URL=http://localhost:5000

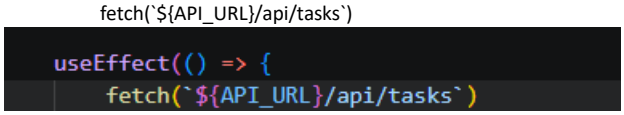




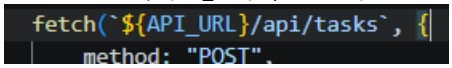
const API_URL = import.meta.env.VITE_API_URL;



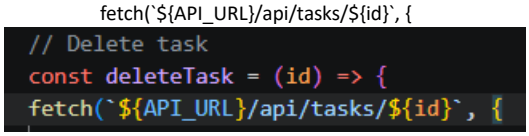
fetch(`${API_URL}/api/tasks`)



fetch(`${API_URL}/api/tasks`, {

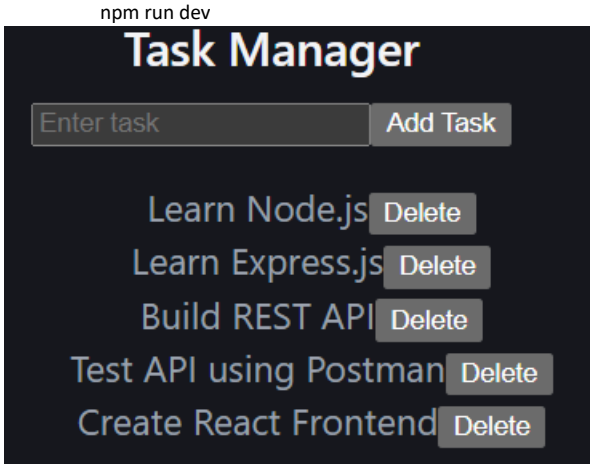


fetch(`${API_URL}/api/tasks/${id}`, {

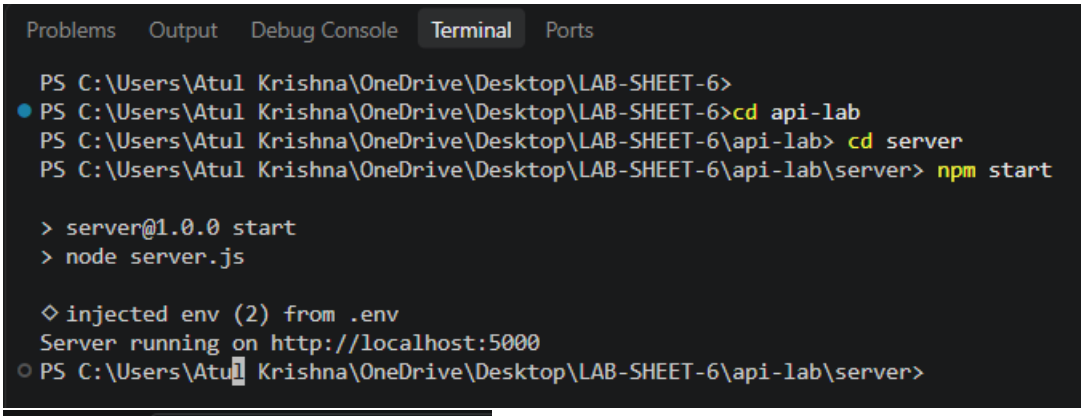


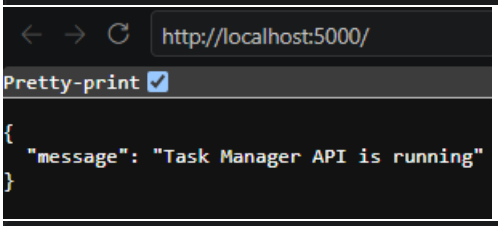
// GET
fetch(`${API_URL}/api/tasks`)
// POST
fetch(`${API_URL}/api/tasks`, {
// DELETE
fetch(`${API_URL}/api/tasks/${id}`, {

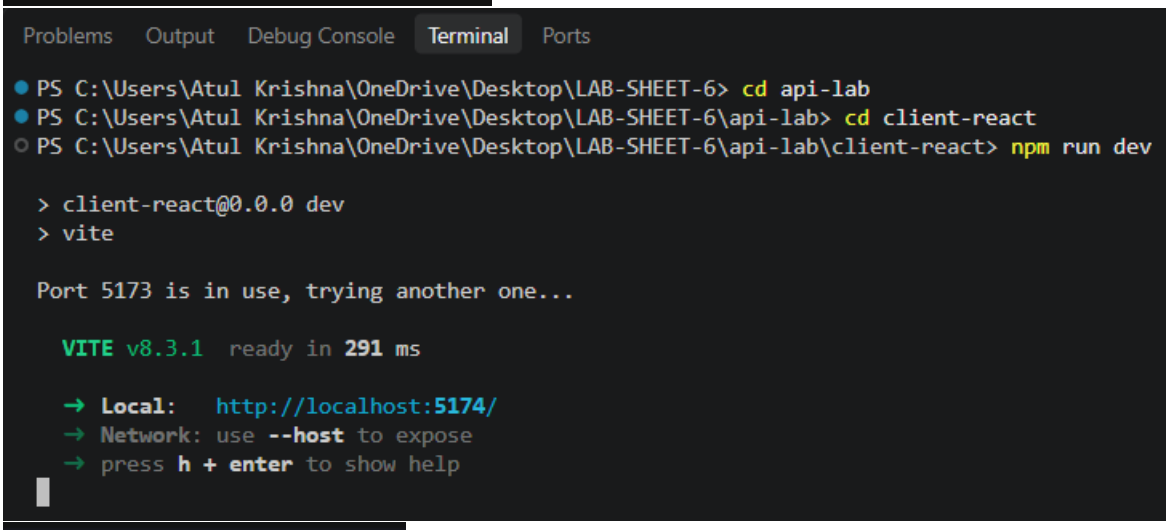
api-lab\client-react
npm run dev

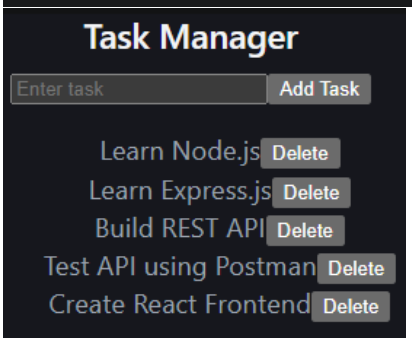


(c) Combine everything into one working “Task Manager Mini App” — the Express API (with middleware, CORS, and error handling) plus the React frontend (with create/delete and loading/error states) — and submit both folders zipped together with your Postman collection export.

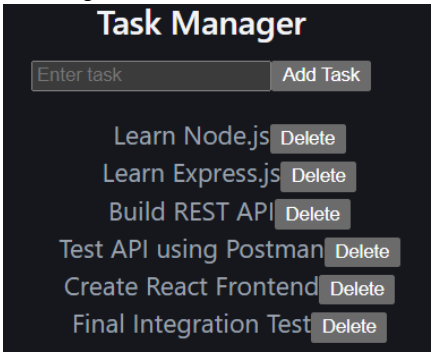




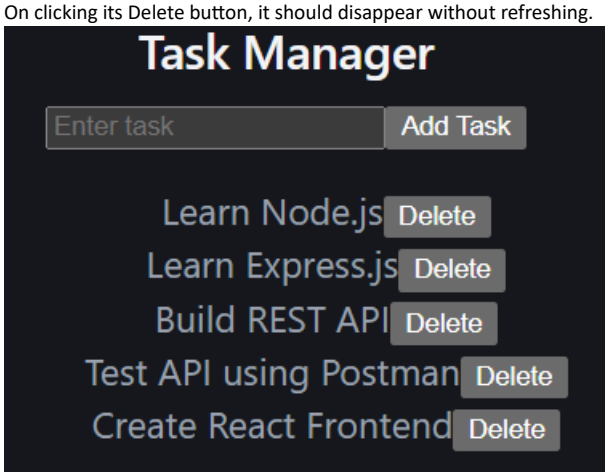




Final Integration Test



On clicking its Delete button, it should disappear without refreshing.




```
if (loading) {
  return <p>Loading...</p>;
}
if (error) {
  return <p>Error: {error}</p>;
}
```

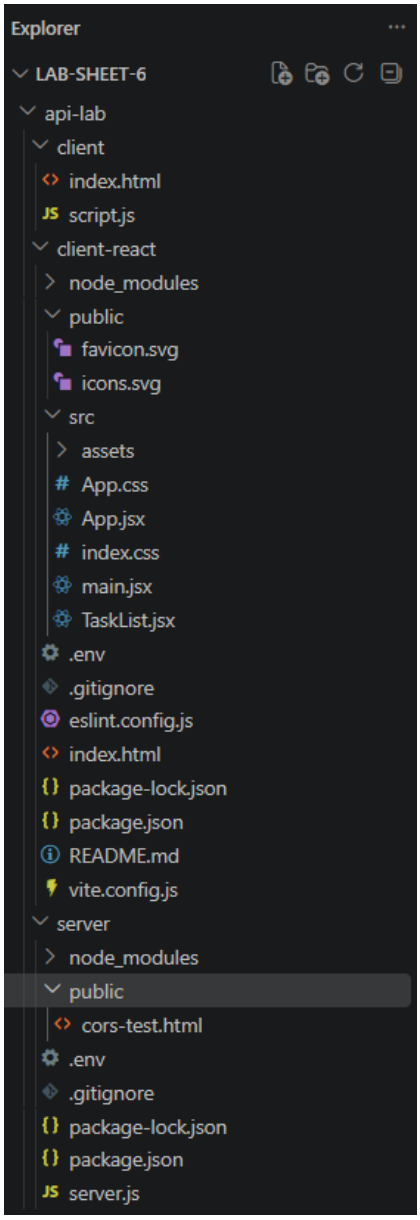
```
if (loading) {
    return <p>Loading...</p>;
}

if (error) {
    return <p>Error: {error}</p>;
}
```

Loading → Error → Data

LAB-SHEET-6

```
└── api-lab
    ├── server
    │   ├── .env
    │   ├── .gitignore
    │   ├── package.json
    │   ├── server.js
    │   └── ...
    │
    ├── client
    │   ├── index.html
    │   └── script.js
    │
    └── client-react
        ├── .env
        ├── package.json
        ├── src
        │   ├── App.jsx
        │   ├── TaskList.jsx
        │   └── ...
        └── ...
```



GET /api/tasks
POST /api/tasks
PUT /api/tasks/:id
DELETE /api/tasks/:id

